



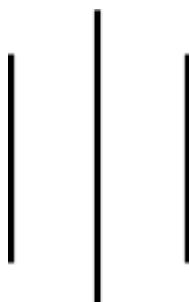
LA GRANDEE INTERNATIONAL COLLEGE

Simalchaur, Pokhara Nepal

A Project Proposal

On

“Voting System”



Submitted to:

Bachelor of Computer Application(BCA) Program

In partial fulfilment of the requirements for the degree of BCA under

Pokhara University

Submitted by:

Name	PU Registration No.	Semester
Ishan Kafle	2024-1-53-0166	IV
Ubika Bajracharya	2024-1-53-0196	IV
Verbina Thapa	2024-1-53-0198	IV

July 6, 2025

Table of Contents

1. Introduction	1
2. Background	2
3. Problem Statement	4
4. Objectives	5
5. Methodology	6
5.1. Requirements Analysis	6
5.2. System Design	6
5.3. Implementation	6
5.4. Database Integration	7
6. System Requirement	8
6.1 Functional Requirements:	8
6.2 Non-Functional Requirements:	8
7. System Design	9
8. Development	12
9. Gantt Chart	13
10. Project result	14
11. Future Enhancements	15
12. Deliverables	16
13. Conclusion	17
14. References	18

List of Figures

Figure 1 level 0 DFD.....	10
Figure 2 level 1 DFD.....	10
Figure 3. level 2 DFD.....	11
Figure 4. Incremental model.....	12
Figure 5. Gantt chart.....	13

1. Introduction

In today's digital age, technology is reshaping the way people interact, communicate, and even vote. A Voting System is a modern method of casting votes, allowing voters to participate in elections from anywhere using digital devices such as computers, tablets, or smartphones. It is designed to improve accessibility, convenience, and efficiency in the electoral process.

Unlike traditional paper-based voting machines that require traditional paper based stuff at voting, the modern voting system enables systematic ease in the candidate choosing process. . This makes it especially beneficial for individuals living with disabilities, or those who are uneducated. . The system typically includes features such as secure voter authentication, encrypted vote transmission, anonymity of votes, and real-time result processing.

2. Background

While working on this project, several problems with existing systems were identified:

1. **Risk of Fraud:** Traditional systems are vulnerable to tampering, such as multiple voting, impersonation, or vote rigging, especially in systems lacking robust verification.
2. **Time-Consuming Counting:** Manual tallying of paper ballots often results in errors, delays in result announcements, and disputes regarding outcome validity.
3. **Lack of Accessibility:** Voters who are unable to be physically present on election day are often excluded from participating.
4. **High Operational Cost:** Traditional elections require extensive resources including paper, polling staff, ballot boxes, and transportation, making the process costly.
5. **Lack of Transparency:** Voters may lose trust in the fairness of the elections due to lack of visibility into how votes are counted or secured.

In recent years, several digital voting systems have emerged to address these shortcomings. Here are some prominent voting software platforms currently available:

- **Helios Voting:** This is an open-source, web-based voting system designed primarily for small-scale elections like university polls and organizational voting. It uses cryptographic techniques to allow voters to verify their votes were counted correctly without revealing who they voted for.
- **Election Buddy:** A commercial solution designed for nonprofits, unions, schools, and other organizations. It features customizable ballots, multi-device access, and real-time result viewing. Election Buddy offers a balance between usability and security.
- **Simply Voting:** This service caters to municipalities, corporations, and educational institutions. It ensures secure access via PINs, emails, and voter ID validation. It provides comprehensive audit trails and detailed analytics.

- **Voatz:** Known for using biometric authentication and blockchain technology, Voatz has been trialed in government pilot programs. It focuses on absentee and overseas voting, providing mobile accessibility with strong security protocols.
- **BallotReady:** While more of a voter information platform, it integrates election tools and resources to help users understand ballot items and plan their voting experience.

Despite the innovations these platforms bring, several challenges persist. Cost remains a barrier for small institutions looking for affordable yet customizable solutions. Moreover, many commercial systems are proprietary, limiting the ability of developers or institutions to adapt the software to specific requirements. Blockchain based systems, while promising, are still in early stages of adoption and raise concerns about scalability and user comprehension.

This project attempts to fill the gap between large-scale, expensive platforms and the need for adaptable, open, and affordable systems. By building the voting system from scratch using open technologies like Java and JavaFX, the focus remains on usability, simplicity, and reliability.

3. Problem Statement

- Manual voting systems were time-consuming, error-prone, and lacked transparency, leading to inefficiencies in vote counting and result declaration.
- Traditional voting processes faced serious security issues such as voter impersonation, vote tampering, and multiple voting attempts.
- Existing digital voting systems were often either too complex for general users or lacked adequate security features.
- There was a clear need for a secure, user-friendly, and efficient online voting system developed in Java to ensure proper voter authentication, safe vote casting, and automated result computation.

4. Objectives

The main objectives of this project are:

- No paper-based or spreadsheet/methods for data recording.
- User friendly UI.
- Secured and automatically managed system for efficient use.

5. Methodology

This section outlines the step-by-step approach used in the design and implementation of the voting System. The project followed a modular and object-oriented development process, utilizing JavaFX for the graphical user interface (GUI) and MySQL for backend data management. The methodology can be broken down into the following stages:

5.1. Requirements Analysis

The initial phase involved gathering and analyzing the functional and non-functional requirements of the system. The key functionalities identified were:

- candidate registration and record management.
- count tracking.
- Secure login system with role-based access.
- A responsive and intuitive GUI.
- Integration with a relational database for data persistence.

5.2. System Design

Using Object-Oriented Programming (OOP) principles, the system was divided into logical components:

- **Data Models** (votersData.java, courseData.java): Represent the structure of entities such as voters and candidates.
- **Controllers** (dashboardController.java, FXMLDocumentController.java): Handle user interaction and control the flow of data between the UI and backend.
- **Database Layer** (database.java, checkcon.java): Establish and manage database connections using JDBC.

The GUI was designed using **FXML** files for layout and **CSS** for styling, promoting separation of concerns.

5.3. Implementation

The implementation phase was carried out using the following tools and technologies:

- **Language:** Java
- **Framework:** JavaFX for UI components
- **Build Tool:** Maven (pom.xml) for dependency and project structure management
- **Database:** MySQL
- **IDE:** IntelliJ IDEA

Each module was developed and tested incrementally:

- The **main application launcher** (voting System.java) initializes the primary dashboard.
- The **login and dashboard UIs** were built using FXML (dashboard.fxml, FXMLDocument.fxml), with associated controllers managing UI logic.
- Data operations such as adding, editing, or deleting candidates records are handled through the dashboardController.java class.

5.4. Database Integration

A MySQL database was designed to store voters count and candidate information. JDBC (Java Database Connectivity) was used to:

- Establish connections (database.java, checkcon.java).
- Perform SQL operations: SELECT, INSERT, UPDATE, DELETE.
- Handle exceptions and ensure stable data flow between the UI and the backend.

packaged as a standalone Java application using Maven. External dependencies (e.g., JDBC driver) were managed using the pom.xml configuration file.

6. System Requirement

6.1 Functional Requirements:

1. Login System:

- Admin login with credentials verification from the database.

2. Candidate Management:

- Add, update, and delete candidate records.
- View the list of candidates with details such as name, email, and picture.

2. Election Management:

- Add, update, and delete election
- Create report.

4. Database Connectivity:

- Use MySQL for database management and continuity.
- Ensure smooth data retrieval and storage using JDBC for database interactions.

6.2 Non-Functional Requirements:

1. Performance:

- The system should handle many records efficiently with quick data retrieval.

2. Usability:

- The interface must be simple and easy for administrators to use

4. Scalability:

- The system must be scalable to accommodate more records (voters, candidate) over time.

7. System Design

The methodology for developing a Voting System involves thorough requirement analysis to understand stakeholder needs, followed by system design focusing on offline operation and user interface. Development entails implementing features like voting system and counting processing, with testing ensuring functionality, usability, and reliability. Deployment includes system installation and staff training, while ongoing maintenance, support, and documentation ensure system sustainability and user satisfaction. Evaluation periodically assesses system effectiveness, guiding future enhancements. This methodology ensures a structured approach to delivering a reliable, user-friendly offline management solution for election department.

7.1 Algorithm

Step 1: Start

Step 2: Initialize database connection

Step 3: Display the Main Menu with options Login and sign up

Step 4: If login is selected direct to login page else if signup is selected direct to new register menu.

Step 5: After login/signup direct to dashboard

Step 6: If admin shows admin dashboard else show user dashboard.

Step 7: Admin create new election with candidate details.

Step 8: Admin create report of the election

Step 9: End

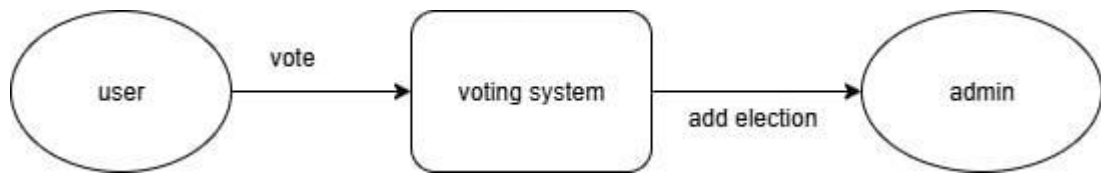


Figure 1 level 0 DFD

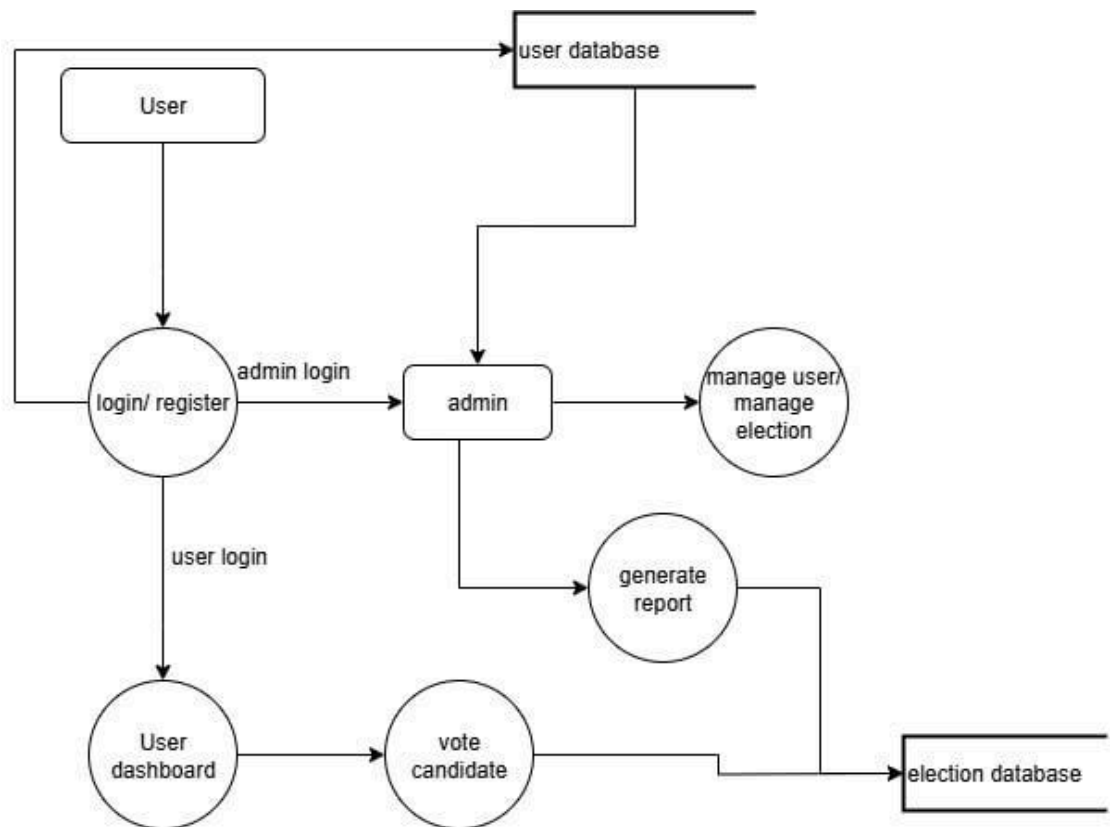


Figure 2 level 1 DFD

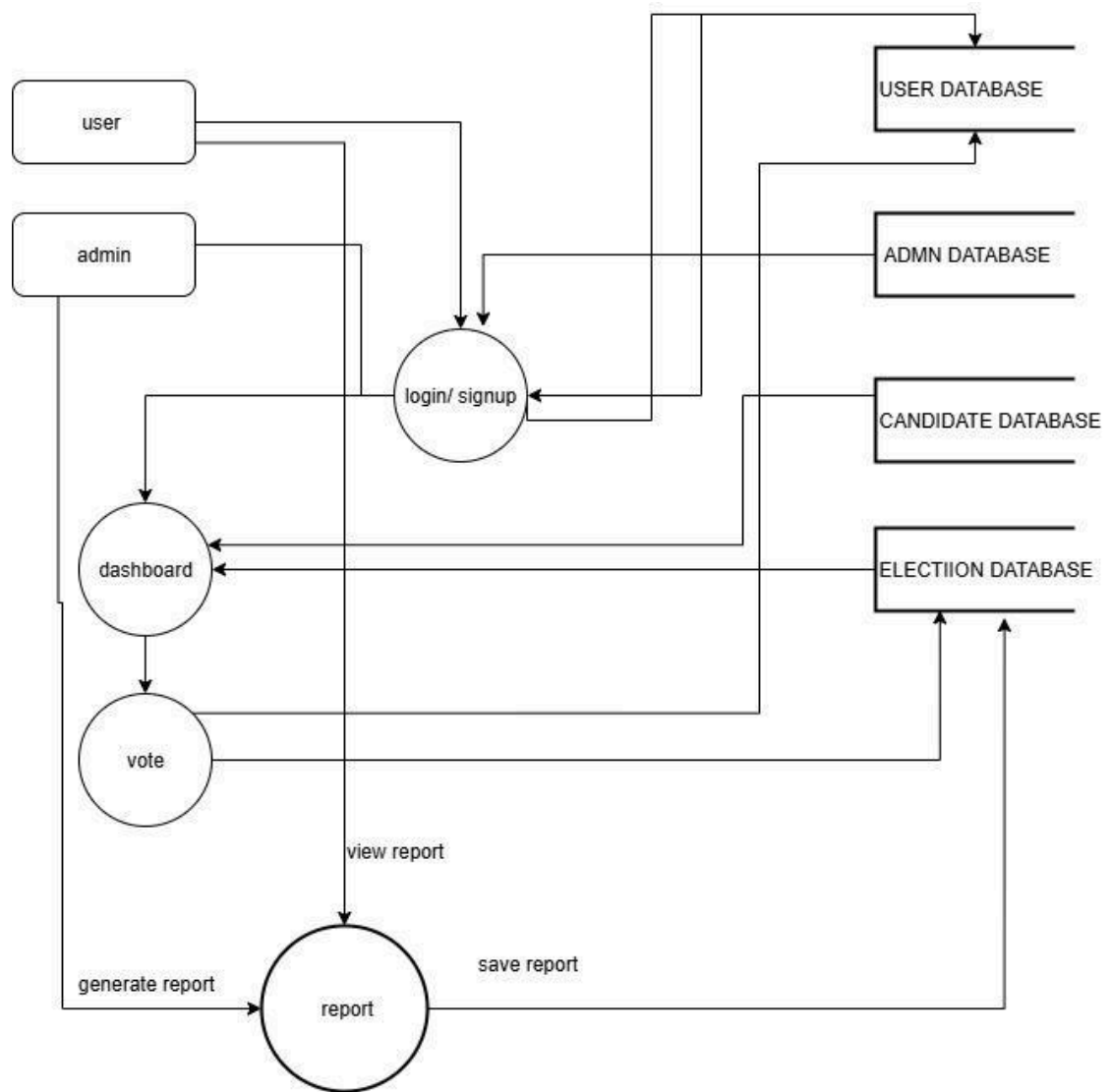


Figure 3. level 2 DFD

8. Development

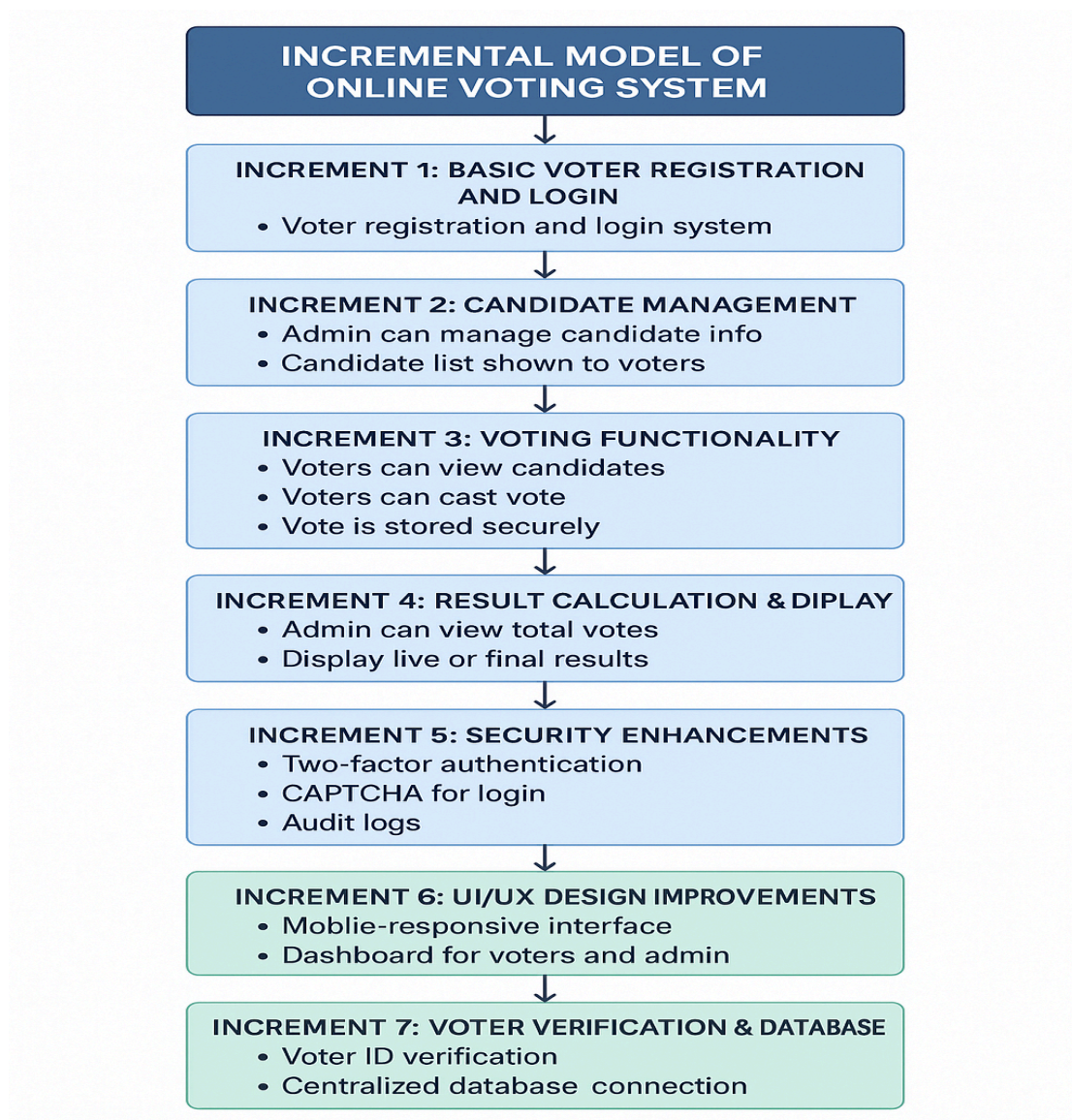


Figure 4. Incremental model

The **Incremental Model** is well-suited for the development of the **Online Voting System**. This approach divides the system into smaller, manageable increments that are developed and delivered in successive stages. Each increment adds specific functionalities—such as voter registration, candidate management, vote casting, and result processing—building upon the previous phase until the full system is complete. This model allows for early testing, user feedback, and continuous improvement, ensuring a reliable and scalable voting solution.

9. Gantt Chart

The detailed time line of the project is given below with the proper timing of mid-defence and final defence.

Timeline: May 14 – July 18 | Midpoint Review: June 15

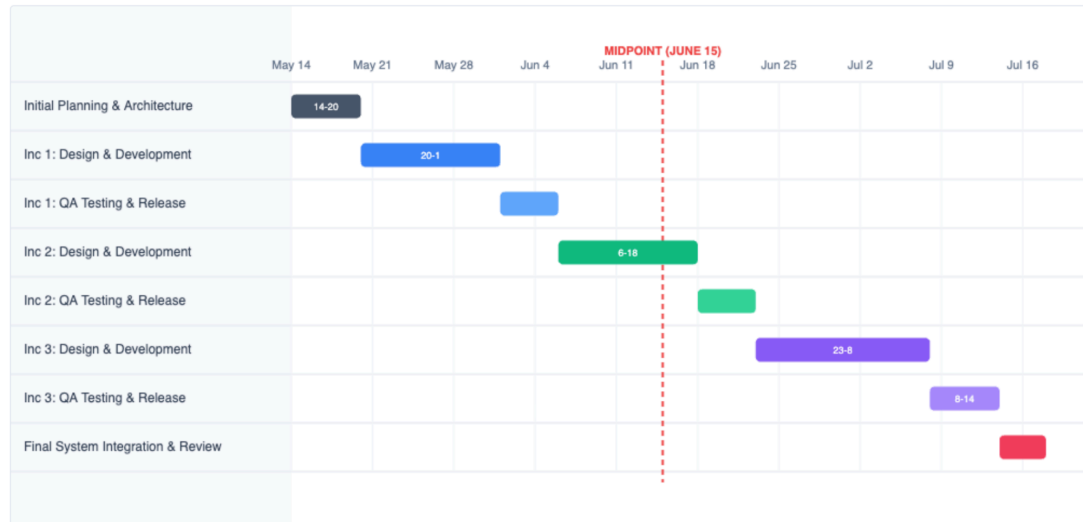


Figure 5. Gantt chart

10. Project result

The project result of the Voting System includes the successful creation of a fully functional and reliable system. The following are the results of our project:

a. Functional System: A fully operational Offline voting System with robust features including candidate management, citizen and eligible person management.

b. High Quality: Each increment undergoes thorough testing, ensuring high quality and reliability through early defect detection and continuous refinement.

c. User Satisfaction: Stakeholders and end-users, including voter, candidate are satisfied with a system that meets their expectations, offers intuitive usability, and enhances voting.

d. Compliance and Security: The system adheres to voting standards for counting security and processing, ensuring compliance and safeguarding candidate and voters.

11. Future Enhancements

Security Enhancements

- Implementation of biometric authentication such as fingerprint or facial recognition to ensure that only authorized users can vote.
- End-to-end encryption for data transmission between client and server to protect user data from external threats.

Data Handling and Analytics

- Real-time vote counting and display of election results through dynamic dashboards and graphical representation.
- Analysis of voter turnout statistics by region, age group, or time to improve future planning.

User Experience Improvements

- Adding multilingual support so voters can select their preferred language for interaction.
- Enhancing accessibility by supporting screen readers, high contrast themes, and adjustable font sizes.

Testing and Auditing

- Provision of mock elections for users to practice voting before the real election.
- Voter feedback module for collecting suggestions, complaints, or experiences after the election.
-

12. Deliverables

1. Fully functional voting System in JavaFX.
2. MySQL database with all necessary tables.
3. User manual for operating the system.
4. Project documentation with explanation of code, features, and setup instructions.

13. Conclusion

The Java-based Voting System project successfully addresses many of the common challenges associated with traditional and manual voting methods. By leveraging core Java technologies along with a user-friendly interface, the system enables secure, efficient, and transparent election processes. The application allows for effective voter authentication, reliable vote casting, and real-time result calculation, significantly reducing the chances of errors, fraud, and administrative burden.

14. References

- (2025). Retrieved from Scene Builder: <https://gluonhq.com/products/scene-builder/>
- (2025). Retrieved from Git & GitHub: <https://git-scm.com/book/en/v2>
- (2025). Retrieved from Java Programming Language:
<https://docs.oracle.com/javase/8/docs/>
- (2025). Retrieved from Stack Overflow & Java Forums: <https://stackoverflow.com>
- (2025). Retrieved from FXML and JavaFX Integration:
https://docs.oracle.com/javase/8/javafx/interoperability-tutorial/fxml_tutorial_intro.html
- (2025). Retrieved from JDBC (Java Database Connectivity):
<https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html>
- (2025). Retrieved from IntelliJ IDEA Documentation:
<https://www.jetbrains.com/idea/documentation/>
- Documentation, O. J. (2022). Retrieved from <https://openjfx.io/>
- GeeksforGeeks, J. a. (2022). Retrieved from
<https://www.geeksforgeeks.org/javafx-mysql-database-connection/>
- Oracle, J. J. (2024). Retrieved from <https://docs.oracle.com/javase/tutorial/jdbc/>
- Report, O. S. (n.d.). Retrieved from
<https://sites.google.com/site/ignoubcafinalyearprojects/project-report/online-voting-system-project-report-1>